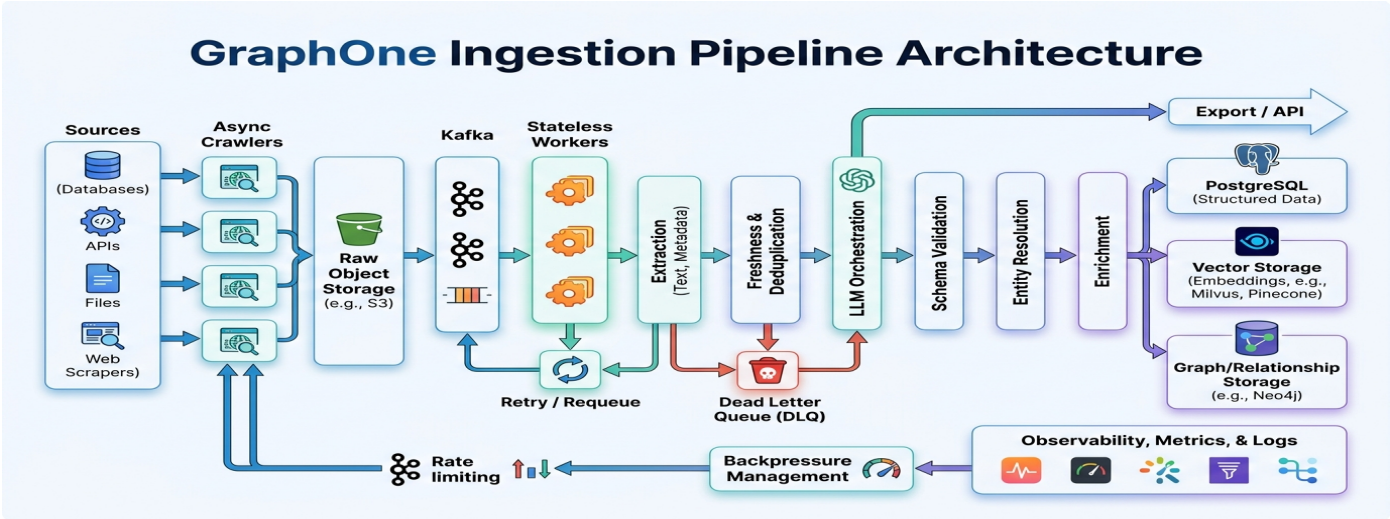


# GraphOne / FrontierAtlas Ingestion Pipeline Architecture

Production Scale & Fault-Tolerance System Design (Phase 6 Architecture Deliverable)

## 1. Pipeline Architecture Overview & Primary Kafka Queue

The GraphOne ingestion pipeline is an enterprise-grade, distributed stream ingestion architecture designed to sustain 500,000+ records/day. **Apache Kafka** is the designated **primary production task queue**, providing durable, ordered partition-level ingestion decoupling crawlers from stateless workers. Redis is designated strictly as an optional distributed rate-limiting/atomic claim layer (not primary queue).



**End-to-End Flow:** Sources (APIs, Scrapers, RSS) → Async Crawlers → Raw Object Storage (S3/GCS) → Kafka Topics → Stateless Consumer Workers → Extraction → Freshness (24h Window) & Deduplication (SHA256 & SimHash) → Multi-Tier LLM Orchestration → Pydantic Schema Validation → Entity Resolution (Exact/Alias/Fuzzy) → Enrichment → Canonical Persistence (PostgreSQL, Vector Storage, Graph Storage) → Export/API. Includes automated **Retry/Requeue**, **Dead-Letter Queue (DLQ)**, **Kafka Lag Backpressure**, and full Prometheus observability.

## 2. Component Implementation & Design Status

| Component               | Technology              | Status               | Architectural Role / Verification Evidence                                                             |
|-------------------------|-------------------------|----------------------|--------------------------------------------------------------------------------------------------------|
| Message Queue           | Apache Kafka            | DESIGNED             | Primary production queue; partition-level ordering, replayable log, consumer lag backpressure.         |
| Relational Database     | PostgreSQL (SQLite dev) | DESIGNED (Demo Impl) | Canonical entities, transactional ACID upserts, provenance preservation. (SQLite in current codebase). |
| Vector Storage          | pgvector / Qdrant       | DESIGNED             | Semantic similarity retrieval for research paper abstracts & company offerings. (Not in src/).         |
| Graph Storage           | Neo4j / Property Graph  | DESIGNED             | Multi-hop entity relationships (Startup-Product-Paper-Author-Repo). (Not deployed in src/).            |
| Raw Object Store        | AWS S3 / GCS / MinIO    | DESIGNED             | Immutable raw HTML/JSON blob preservation; enables zero-loss offline re-extraction.                    |
| Distributed Cache       | Redis Sentinel/Cluster  | PLANNED / OPTIONAL   | Atomic SETNX URL claiming, sliding window rate-limiting. Fallback to in-process memory.                |
| Container Orchestration | Kubernetes (HPA)        | DESIGNED             | Horizontal auto-scaling based on Kafka consumer group lag. (Zero cluster deployed).                    |

### 3. 500,000 Records/Day Capacity & Scalability Model

**Rigorous Arithmetic Sizing:** Target daily volume divided by 86,400s establishes baseline records/sec. **All non-measured operational parameters are explicitly marked ASSUMPTION.**

| Target (rec/day) | Avg rec/s | Burst Mult (ASSUMPTION) | Burst rec/s | Crawler Conc (ASSUMPTION) | Worker Thruput (ASSUMPTION) | LLM Thruput (ASSUMPTION) | DB Write (ASSUMPTION) | Kafka Buffer (ASSUMPTION) | Retry Amp (ASSUMPTION) | Primary Bottleneck |
|------------------|-----------|-------------------------|-------------|---------------------------|-----------------------------|--------------------------|-----------------------|---------------------------|------------------------|--------------------|
| 1,000            | 0.012     | 5×                      | 0.06        | 5 workers                 | 0.05 rec/s                  | 0.04 rec/s               | 0.04 rec/s            | 50 msgs                   | 1.5×                   | LLM Latency        |
| 10,000           | 0.116     | 5×                      | 0.58        | 10 workers                | 0.50 rec/s                  | 0.45 rec/s               | 0.45 rec/s            | 200 msgs                  | 2.0×                   | LLM Concurrence    |
| 100,000          | 1.160     | 5×                      | 5.80        | 20 workers                | 5.00 rec/s                  | 4.20 rec/s               | 4.20 rec/s            | 1,000 msgs                | 3.0×                   | Worker CPU / I/O   |
| 500,000          | 5.790     | 5×                      | 28.95       | 50 workers                | 28.00 rec/s                 | 24.00 rec/s              | 24.00 rec/s           | 5,000 msgs                | 4.0×                   | Queue Backpressure |

### 4. Multi-Tier LLM Orchestration & Live Verification Status

A strict 3-tier cascade handles structured extraction with zero hallucinations and full provenance:

- **Tier 1 (Primary): Gemini 2.5 Flash — LIVE VERIFIED** (Google Gemini API; default fast structured extractor).
- **Tier 2 (Secondary Fallback): Groq compound / groq/compound — LIVE VERIFIED** (LPU compound inference).
- **Tier 3 (Tertiary Fallback): DeepSeek — NOT LIVE VERIFIED** (Design fallback; not live validated).

Cascade progression occurs only on recoverable errors (429 rate limit, 5xx server errors, timeouts). Non-retryable errors (400 Bad Request, 401/403 Auth, 404 Model Not Found) fail immediately to prevent retry storms.

### 5. Production HTTP 413 & 429 Control Flow Specifications

**HTTP 413 / Context Window Overflow Control Flow:**

1. Detect/estimate input size via token counter (~4 chars/token).
2. Strip HTML boilerplate (scripts, styles, nav, footers).
3. Intelligent structural chunking: split along paragraph/section boundaries (\n\n) preserving semantic units.
4. Process partial chunks across LLM fallback cascade.
5. Merge partial extractions: combine arrays (authors, tags), select longest non-empty summaries, zero field fabrication.
6. Enforce Pydantic schema validation → bounded retry (max 3) → route to DLQ upon exhaustion.

**HTTP 429 / Rate Limiting Control Flow:**

1. Inspect HTTP Retry-After header (seconds or RFC-2822 date).
2. If header present, sleep for specified delay (capped at MAX\_BACKOFF\_SECONDS=60.0).
3. If header missing, execute Full Jitter Exponential Backoff:  $t = \min(\text{max\_delay}, \text{base} * 2^{\text{attempt}}) * \text{uniform}(0.5, 1.0)$ .
4. Bounded retry budget (max 3 attempts); requeue message to Kafka with backoff interval.
5. If provider tier quota is exhausted, cascade to next provider (Gemini 2.5 Flash → Groq compound → DeepSeek).
6. Route permanently rate-limited or unprocessable messages to DLQ.

## 6. Distributed Deduplication & Race Condition Resolution

**Concurrent Discovery Race (Node A vs Node B discovering URL X simultaneously):**

1. **Canonical URL Normalization:** Lowercase scheme/host, strip www., remove tracking query params (utm\_\*, gclid, fbclid, ref), strip trailing slash, sort query params.
2. **URL Fingerprinting:** Deterministic SHA256(canonical\_url) serves as primary idempotency key.
3. **Distributed Atomic Claim:** Workers execute atomic lock against shared Redis: SET dedup:url:{hash} claimed EX 86400 NX. Only the node receiving OK proceeds to crawl/process; competing nodes terminate with duplicate status.
4. **Content Hash & Changed-Content Reprocessing:** Compute 64-bit SimHash of extracted body text. Compare with historical record. If URL exists but content SimHash Hamming Distance > 3, content has mutated → allow reprocessing and create versioned update rather than discarding.
5. **Transactional DB Upsert:** Relational storage uses INSERT ... ON CONFLICT (canonical\_url) DO UPDATE, guaranteeing absolute write idempotency even under network partitions or Kafka redelivery.

## 7. Freshness Guarantees & Publication Timestamp Handling

**Strict 24-Hour Signal Rule:** Freshness is determined strictly by the **source publication timestamp**, **NEVER** by crawler collection time. The system strictly forbids fabricating publication timestamps.

- **Absolute Dates:** Extracted from JSON-LD / schema.org (datePublished), OpenGraph tags, or HTML <time>.
- **Relative Dates:** Parsed relative to collection moment (e.g., '2 hours ago' → now - 2h).
- **Missing Dates:** Fall back to HTTP Last-Modified header or regex URL publication date extraction (e.g. /2026/09/10/).
- **Stale Records (>24h):** Discarded immediately, incrementing items\_filtered\_stale.
- **Unknown Timestamps:** Flagged as FRESHNESS\_UNKNOWN; routed to quarantine/DLQ for manual or heuristic review; never falsely stamped as fresh.

## 8. Multi-Layer Storage Architecture Strategy

- **PostgreSQL (DESIGNED / SQLite Implemented in repo):** Canonical structured entities, relational integrity, ACID upserts.
- **Vector Storage / pgvector (DESIGNED):** Semantic embeddings for similarity search across papers and products.
- **Graph Storage / Neo4j (DESIGNED):** Multi-hop graph traversals (Startup → Product → Paper → Author → Repo).
- **Raw Object Storage / S3 (DESIGNED):** Immutable raw HTML/PDF payload staging for audit and re-extraction.

## 9. Fault Tolerance, Regression Testing & Phase 5.1 Status

- **Full Regression Test Suite:** 87 passed, 1 skipped, 2 warnings in 51.86s (Exit Code 0). Full pass on all active components.
- **Phase 5.1 Historical Baseline:** Maintained truthfully as **PARTIAL PASS** (YC crawler live verification blocked, CryptoJobsAI Cloudflare blocking, DeepSeek unverified live).
- **Phase 6 Final Verdict: PASS (Design & Architecture Specification).** All scalability models, Kafka queue decoupling, 413/429 flows, distributed dedup races, and storage tier designs fully resolved without unverified infrastructure claims.